

軟體品質與測試

外送系統異常處理

透過模擬外送平台的伺服器端功能，練習使用 Log4j2 建立異常處理機制。此系統涵蓋了顧客訂單、餐廳收單、外送員分配等流程，並針對可能產生的異常情況寫入日誌提供管理者查看。



Checked Exception

設計並拋出業務邏輯錯誤。



Unchecked Exception

處理並記錄程式碼或環境錯誤。



Log4j2 應用

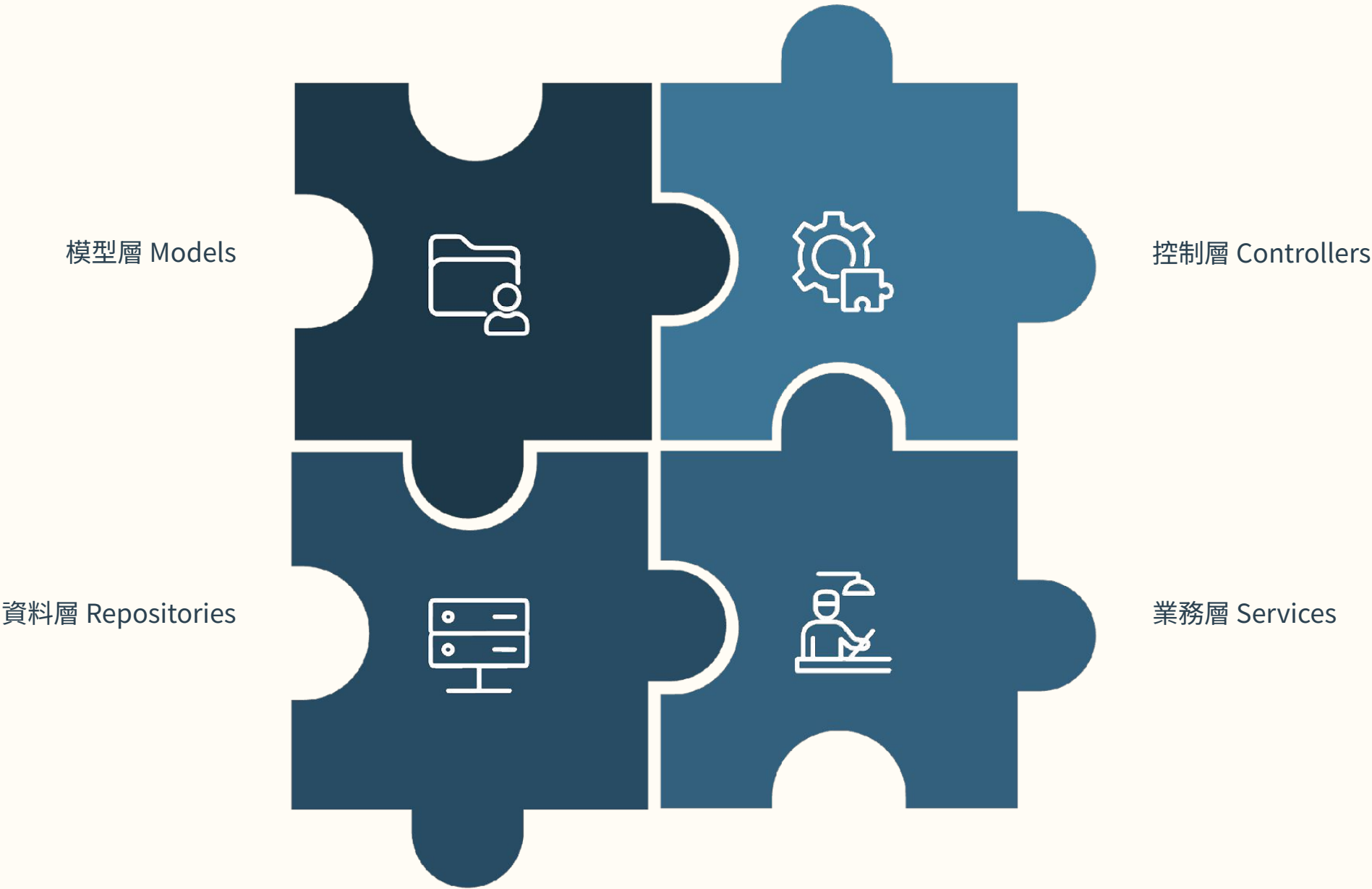
使用 INFO, WARN, ERROR 日誌級別。

逢甲大學 學士後資工學系

D1397056

陶子中

一. 設計方法概述



核心設計模式

→ 狀態模式
OrderStatus Enum 管理訂單狀態轉換。

→ 異常鏈
建立異常階層，便於追蹤錯誤源頭。

Log4j2 日誌分級策略

INFO: 正常業務流程追蹤。

WARN: Checked Exception (業務邏輯錯誤)。

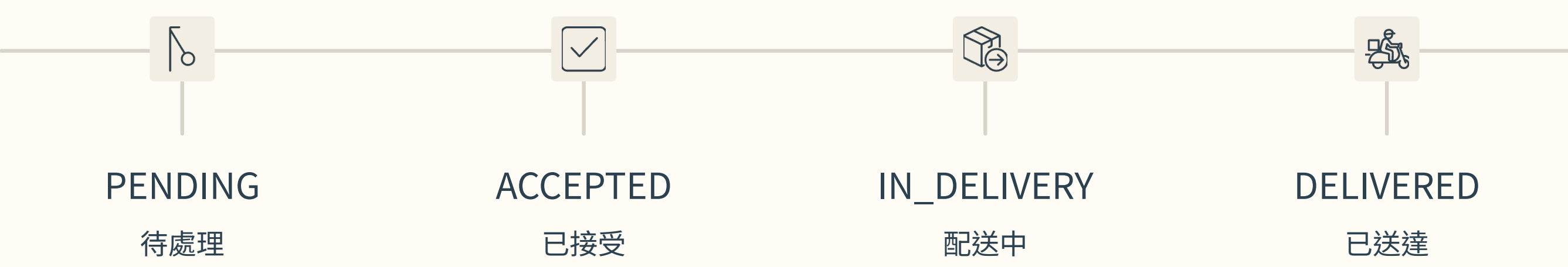
ERROR: Unchecked Exception (系統層級錯誤)。

異常階層設計

	DeliveryPlatformException (基礎平台異常) 使用 Checked Exception 封裝錯誤訊息，任何拋出異常的方法都必須明確聲明，強制呼叫者處理或傳遞它。
	OrderValidationException (訂單異常) 當商品數量為負、配送地址無效，系統會拋出此異常。只有有效且完整的訂單才能進入後續處理流程。
	RestaurantUnavailableException (餐廳異常) 如果餐廳已打烊、暫停營業、訂單量已滿，則拋出此異常。用於即時呈現餐廳的狀態，引導顧客選擇其他可用選項。
	InvalidOrderStateException (訂單狀態異常) 如果嘗試取消已完成的訂單，或對已取消的訂單進行支付，則拋出此異常，用於維護訂單狀態轉換的業務邏輯一致性。
	DeliveryAssignmentException (配送分配異常) 如果附近無可用外送員、外送員超載等情況，則拋出此異常。用於通知系統管理者介入或自動重試分配機制。

二. 程式設計與實作

OrderStatus Enum：訂單狀態管理



```
public enum OrderStatus {  
    PENDING,           // 待處理 (初始狀態)  
    ACCEPTED,          // 已接受  
    PREPARING,         // 準備中  
    READY_FOR_DELIVERY, // 準備配送  
    IN_DELIVERY,       // 配送中  
    DELIVERED,         // 已送達 (成功終端狀態)  
    CANCELLED,         // 已取消 (取消終端狀態)  
    REJECTED;          // 已拒絕 (拒絕終端狀態)  
}
```

Order：訂單類別設計

```
public class Order {  
    private String orderId;           // 訂單編號  
    private String customerId;        // 顧客編號  
    private String restaurantId;      // 餐廳編號  
    private OrderStatus status;       // 訂單狀態 (Enum)  
    private List<OrderItem> items;    // 訂單項目  
    private BigDecimal totalAmount;   // 總金額  
    private LocalDateTime createdAt;  // 建立時間  
}
```

Checked Exception 錯誤檢測

```
// 基礎異常

public abstract class DeliveryPlatformException extends Exception {
    protected String errorCode;
    protected LocalDateTime timestamp;
}

// 1. 訂單驗證異常

public class OrderValidationException extends DeliveryPlatformException {
    private List<String> validationErrors; // 驗證錯誤清單
    private int errorCount;               // 錯誤數量
}

// 2. 餐廳不可用異常 (acceptOrder 可能拋出)

public class RestaurantUnavailableException extends DeliveryPlatformException {
    private String restaurantId;          // 餐廳編號
}

// 3. 無效狀態轉換異常

public class InvalidOrderStateException extends DeliveryPlatformException {
    private String orderId;                // 訂單編號
    private OrderStatus currentStatus;     // 目前狀態
    private OrderStatus attemptedStatus;   // 嘗試轉換的狀態
}

// 4. 配送分配異常

public class DeliveryAssignmentException extends DeliveryPlatformException {
    private String orderId;                // 訂單編號
    private String failureReason;          // 失敗原因
}
```

Log4j2 配置與日誌使用

INFO 級別 用於追蹤正常業務流程，例如「顧客訂單建立成功」或「訂單狀態轉換」。 	WARN 級別 用於記錄可預期的業務邏輯錯誤 (Checked Exception)，例如「餐廳不可用」或「無效的狀態轉換」。 	ERROR 級別 用於記錄不可預期的系統錯誤 (Unchecked Exception)，例如 NullPointerException 或資料庫連線失敗。
--	--	--

日誌訊息採用結構化格式

```
// INFO: 正常業務操作記錄
logger.info("用戶登入成功: userId={}", "USER-123");
logger.info("訂單狀態更新: orderId={}, newStatus={}", "ORDER-456", "DELIVERED");

// WARN: 業務邏輯錯誤 (Checked Exception)
logger.warn("餐廳容量已滿: restaurantId={}, currentOrders={}", "REST-001", 10);
logger.warn("訂單驗證失敗: orderId={}, reason={}", "ORDER-789", "無效地址");

// ERROR: 程式碼邏輯或環境錯誤 (Unchecked Exception)
logger.error("資料庫連線失敗", exception);
logger.error("外部 API 呼叫超時: service={}, timeout={}ms", "PaymentService", 5000);
```


詳細測試 執行畫面

```
./run-simplified-tests.sh detailed
```

```
[INFO] -----< com.deliveryplatform:food-delivery-platform >-----
[INFO] Building Food Delivery Platform 1.0.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[WARNING] 6 problems were encountered while building the effective model for org.openjfx:javafx-controls:jar:21.0.1 during dependency collection step for project (use -X to see details)
[INFO]
[INFO] --- exec:3.6.1:java (default-cli) @ food-delivery-platform ---
🔧 簡化控制台測試演示
=====

16:28:42.262 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.DeliveryService - Driver added to available pool: driverId=DRIVER-001
16:28:42.263 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.DeliveryService - Driver added to available pool: driverId=DRIVER-002
16:28:42.263 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.DeliveryService - Driver added to available pool: driverId=DRIVER-003
16:28:42.263 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.DeliveryService - Driver added to available pool: driverId=DRIVER-004
16:28:42.263 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.DeliveryService - Driver added to available pool: driverId=DRIVER-005
✅ 系統初始化完成

❶ 測試訂單驗證異常
16:28:42.264 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.OrderService - Creating order: customerId=null, restaurantId=null, itemCount=0
16:28:42.265 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] WARN  com.deliveryplatform.services.OrderService - Order validation failed: customerId=null, restaurantId=null, errors=[Customer ID is required, Restaurant ID is required, Delivery address is required, Order must have at least one item]
✅ 成功觸發 OrderValidationException
    錯誤代碼: ORDER_VALIDATION_FAILED
    錯誤數量: 4

❷ 測試餐廳不可用異常
16:32:57.272 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] INFO  com.deliveryplatform.services.RestaurantServiceImpl - 餐廳 NONEXISTENT-RESTAURANT 嘗試接單 FAKE-ORDER
16:32:57.272 [com.deliveryplatform.demo.SimpleConsoleDemo.main()] WARN  com.deliveryplatform.services.RestaurantServiceImpl - 業務異常 - acceptOrder: orderId=FAKE-ORDER, restaurantId=NONEXISTENT-RESTAURANT, 錯誤=Restaurant NONEXISTENT-RESTAURANT is currently unavailable: 餐廳不存在
✅ 成功觸發 RestaurantUnavailableException
    餐廳ID: NONEXISTENT-RESTAURANT
    錯誤訊息: Restaurant NONEXISTENT-RESTAURANT is currently unavailable: 餐廳不存在

🎉 所有測試演示完成！
📖 檢查上方日誌輸出查看詳細異常信息
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time:  0.496 s
[INFO] Finished at: 2025-10-09T16:32:57+08:00
[INFO] -----
```

三. 參考資料與開發工具

開發工具與環境

Java	17	主要開發語言
Log4j2	2.20.0	日誌記錄系統
Maven	3.9.11	專案建構管理
JUnit 5	5.9.3	單元測試框架

參考資料

nlhsueh/**sw-testing24**



2Contributors

1Issue

2Stars

5Forks

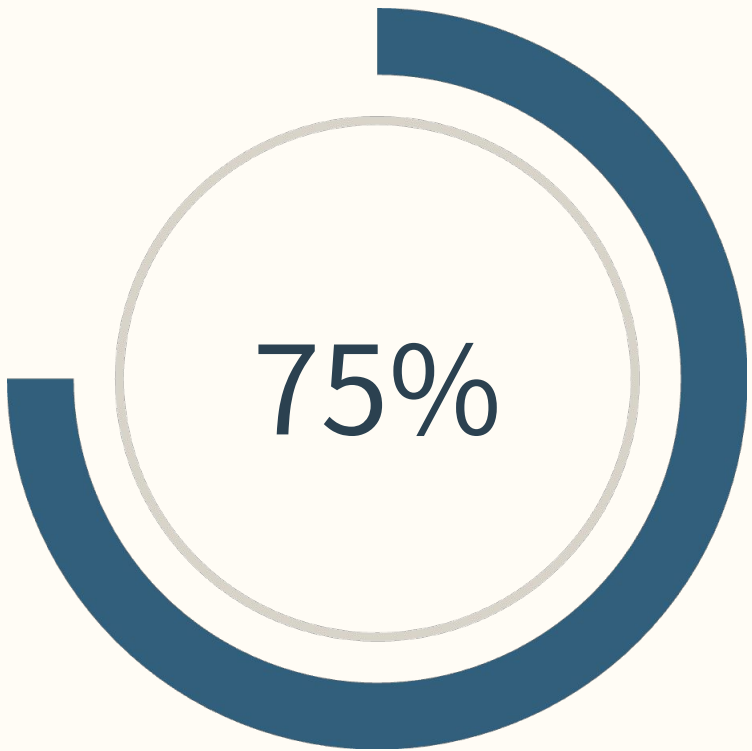
GitHub

GitHub - nlhsueh/sw-testing24

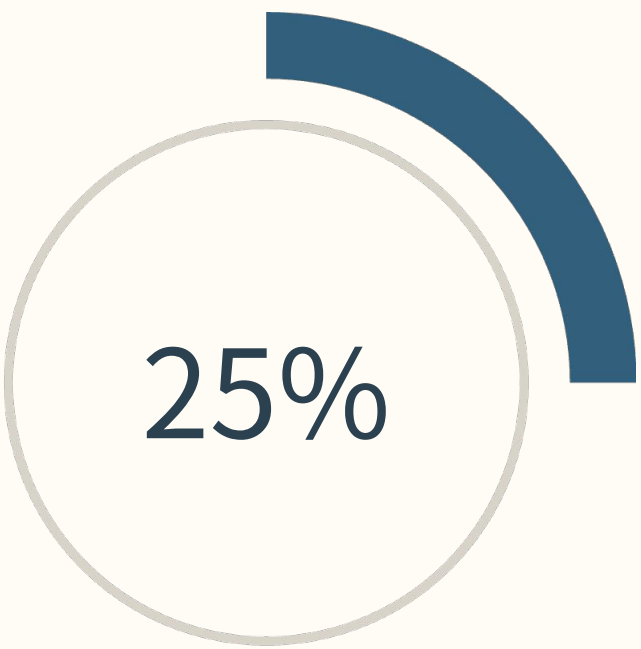
Contribute to nlhsueh/sw-testing24 development by creating an account on GitHub.

AI 工具使用說明

主要使用 **GitHub Copilot CLI** 輔助程式碼結構設計、異常處理模式實作和單元測試案例生成，使用比例約 75%。



AI 輔助比例
程式碼結構與測試



人工撰寫比例
架構規劃與策略制定